Assignment -2

Problem 1: The Highland Water Grid (12 Marks)

Problem Statement

You are a civil engineer hired by a regional government to design a water-sharing pipeline network for N mountain villages, so that water can be redirected between any two villages during dry seasons.

You are given a list of candidate pipeline routes between pairs of villages, each with a construction cost. However, there are two complications:

a. **Legacy links:** Some pairs of villages are already connected through an old aqueduct system that still functions. Connecting such a pair costs 0.
b. **Treaty obligation:** Under a cross-border water-sharing treaty, a specific pipeline between Village X and Village Y MUST be built, regardless of its cost. Even if X and Y are already connected some other way.

PART-A:

Find the minimum total cost required so that water can flow between every pair of villages, directly or indirectly.

Input Format

⇒ **Line 1:** three integers N, M, A → the number of villages, the number of candidate pipeline routes, and the number of existing links.

⇒ **Next A lines**: two integers u v → a pair of villages already linked by a link (cost 0).

⇒ **Next line**: three integers X Y C → the mandatory treaty pipeline between villages X and Y, and its construction cost C.

⇒ **Next M lines**: three integers u v cost → a candidate pipeline route between villages u and v with the given construction cost.

Output Format

Print a single integer: the minimum total cost to connect all N villages.

Sample Test Case

Sample Input

```
6 6 1    // N M A
1 2      // u v
3 4 15   // X Y C
1 3 10
2 5 8
4 6 12
5 6 4
1 4 20
3 5 7
```

Sample Output

```
34
```

Explanation

Villages 1 and 2 are already linked for free via the existing link.

The treaty pipeline between 3 and 4 must be built regardless of cost: cost so far = 15.

Remaining candidate routes, sorted by cost: (5,6)=4, (3,5)=7, (2,5)=8, (1,3)=10, (4,6)=12, (1,4)=20.

- Add (5,6), cost 4 → connects {5} and {6}. Running total = 19.
- Add (3,5), cost 7 → connects {3,4} and {5,6}. Running total = 26.
- Add (2,5), cost 8 → connects {1,2} and {3,4,5,6}. Running total = 34.

All 6 villages are now in a single component, using exactly 5 connecting edges (1 free + 1 mandatory + 3 chosen). No cheaper combination connects everyone, so the minimum total cost is 34.

Part B (Follow-up) — Budget-Constrained Grid (8 Marks)

The government now imposes a strict construction budget B. **The free links and the mandatory treaty pipeline (From Part A) are unaffected by the budget** and are still built as before. For the remaining candidate routes, keep adding them as long as doing so does not push the total spending above B. As soon as adding the next (cheapest remaining) route would exceed B, stop adding routes entirely.

Villages that end up in separate, disconnected clusters after this process will each be equipped with an independent solar desalination plant, so every cluster can produce its own safe drinking water.

Task: Given the same village/route data (as Part A) plus a budget B, find the minimum number of desalination plants needed.

***Hint**: Now, there will be multiple spanning trees. So, in kruskal, there should be multiple roots if the budget is maintained.*

Input Format

Line 1: four integers N, M, A, B ⇒ as in Part A, plus the construction budget B.

Next A lines: two integers u v ⇒ already-linked villages.

Next line: three integers X Y C ⇒ the mandatory treaty pipeline and its cost.

Next M lines: three integers u v cost ⇒ a candidate pipeline route and its cost.

Output Format

Print a single integer: the minimum number of desalination plants required.

Sample Input

```
6 6 1 30   // N M A B
1 2  // u v
3 4 15 // X Y C
1 3 10
2 5 8
4 6 12
5 6 4
1 4 20
3 5 7
```

Sample Output

2

Explanation

As before, villages 1–2 are linked free, and the mandatory pipeline (3,4) costs 15, bringing the running total to 15.

Sorted remaining routes: (5,6)=4, (3,5)=7, (2,5)=8, (1,3)=10, (4,6)=12, (1,4)=20.

- Add (5,6): total = 19 (≤ 30) → clusters {1,2}, {3,4,5,6}.
- Add (3,5): total = 26 (≤ 30) → merges the 5,6 side with the 3,4 side; clusters {1,2}, {3,4,5,6}.
- Try (2,5): total would become 34 (> 30) → stop.

Two clusters remain: {1, 2} and {3, 4, 5, 6}. So 2 desalination plants are needed.

Problem 2: The Data Packet Relay (8 Marks)

Problem Statement

You are optimizing the routing layer of a cloud provider's network. The network has N servers (numbered 1 to N) connected by M direct network links. Each link has a fixed transmission latency.

Because of ongoing congestion, **every server i also has an average delay proc_delay[i]:** the time a packet spends waiting inside that server before it can be forwarded onward (or, if it is the final server, before it is considered delivered).

A client's request originates at Server 1 and must reach the destination data center at Server N.

Task

Find the minimum total time: **transmission latency+processing delay at every server the packet arrives at (including the destination, but not the origin server)**, for a packet to travel from Server 1 to Server N.

Input Format

Line 1: two integers N M ⇒ the number of servers and the number of network links.

Next M lines: three integers u v w ⇒ an undirected link between servers u and v with transmission latency w.

Last line: N space-separated integers proc_delay[1] proc_delay[2] ... proc_delay[N] ⇒ the processing delay at each server.

Output Format

Print a single integer: the minimum total time for a packet to travel from Server 1 to Server N.

Sample Test Case

Sample Input

6 9
1 2 4
1 3 2
2 3 1
2 4 5
3 4 8
3 5 12
4 6 6
5 6 2
4 5 3
0 3 1 2 4 2

Sample Output

21

Explanation

The processing delays are: server 1→0, server 2→3, server 3→1, server 4→2, server 5→4, server 6→2 (server 1's own delay is never charged, since the packet starts there).

The optimal route is 1 → 3 → 4 → 6:

- 1 → 3: latency 2 + processing at 3 (1) = 3. Running total = 3.
- 3 → 4: latency 8 + processing at 4 (2) = 10. Running total = 3+10=13.
- 4 → 6: latency 6 + processing at 6 (2) = 8. Running total = 13+8=21.

Every other route (for example 1 → 2 → 4 → 6, or 1 → 3 → 5 → 6) costs 22 or more, so the minimum total time is 21.